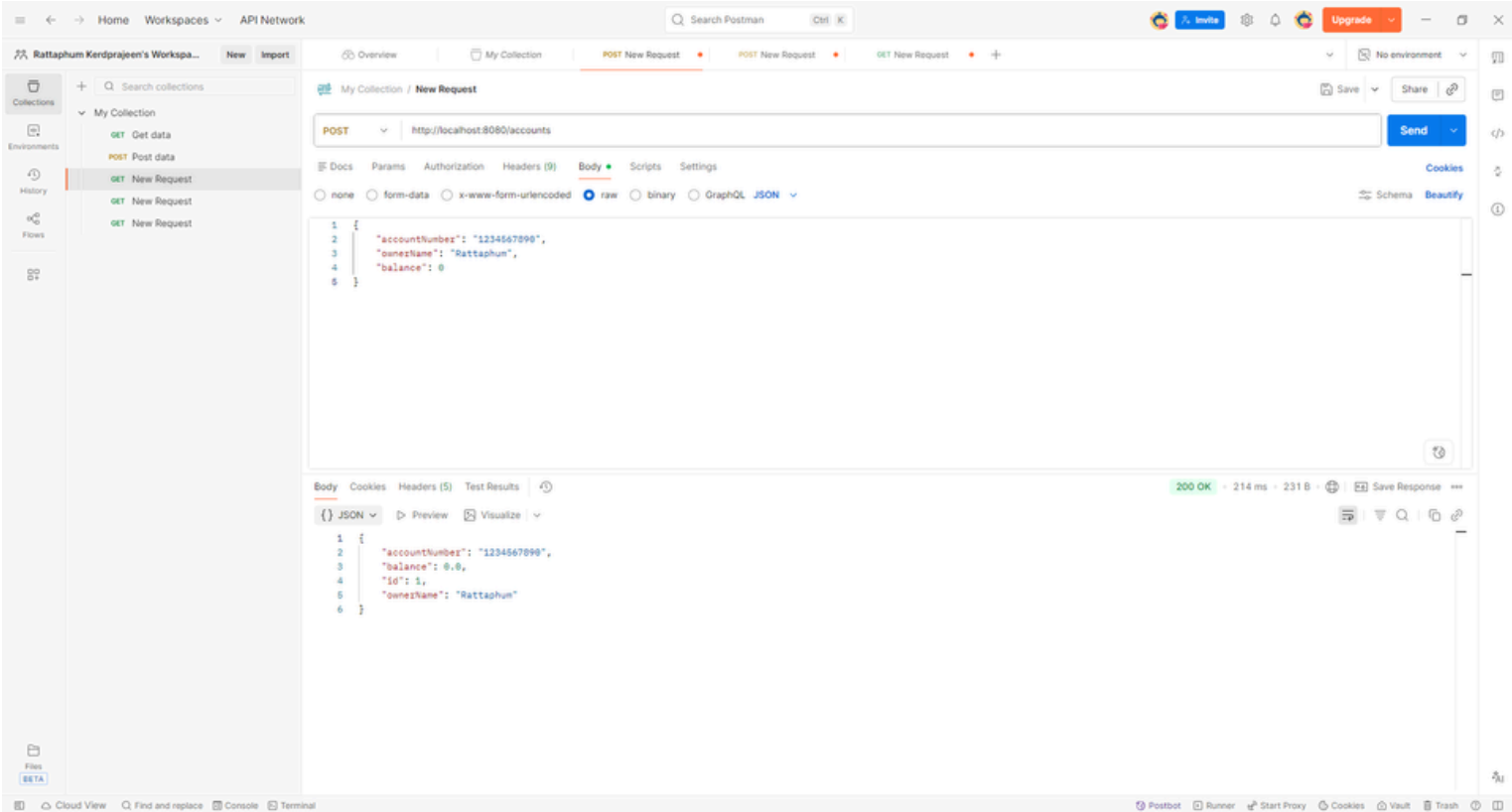
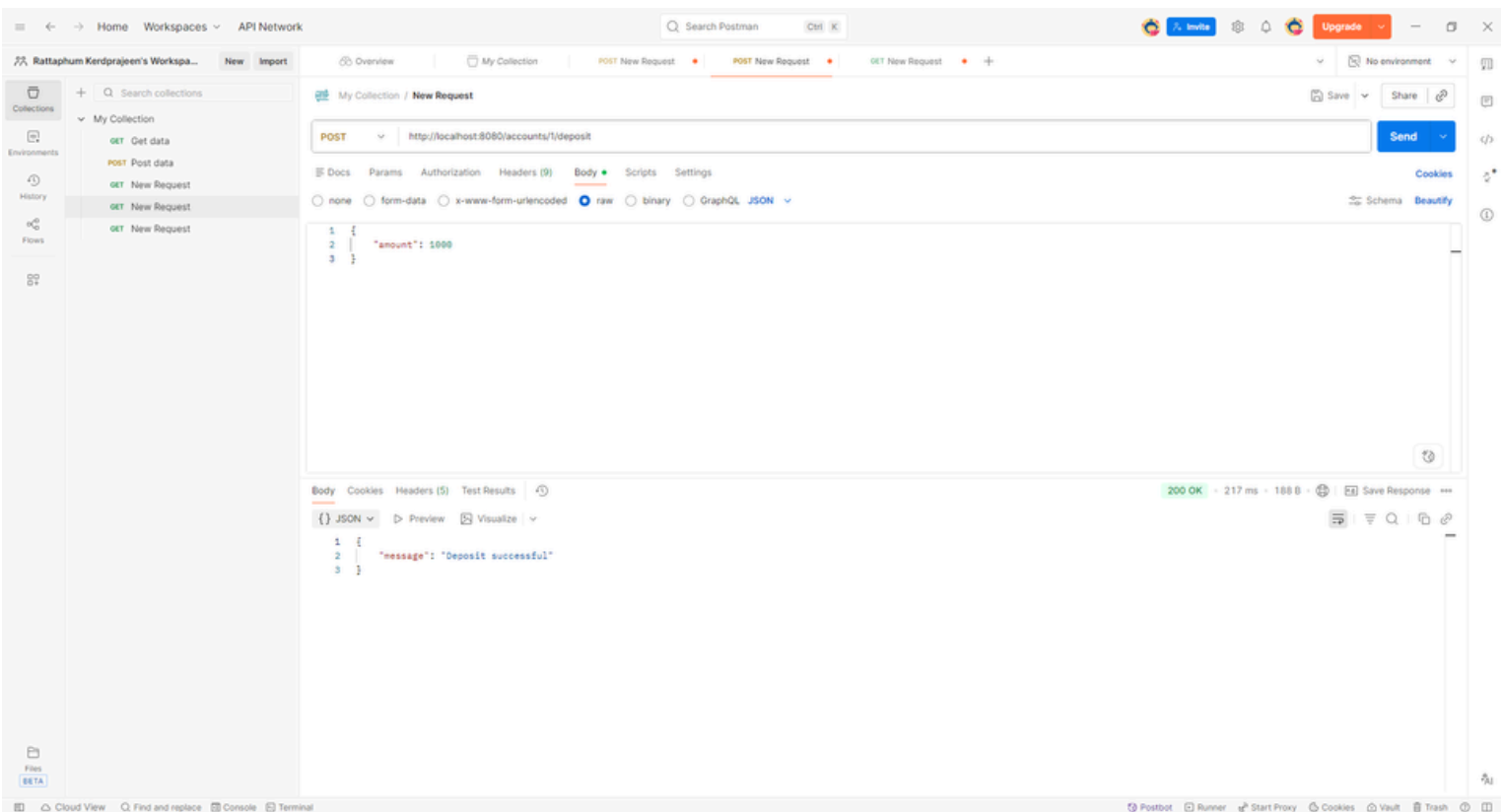
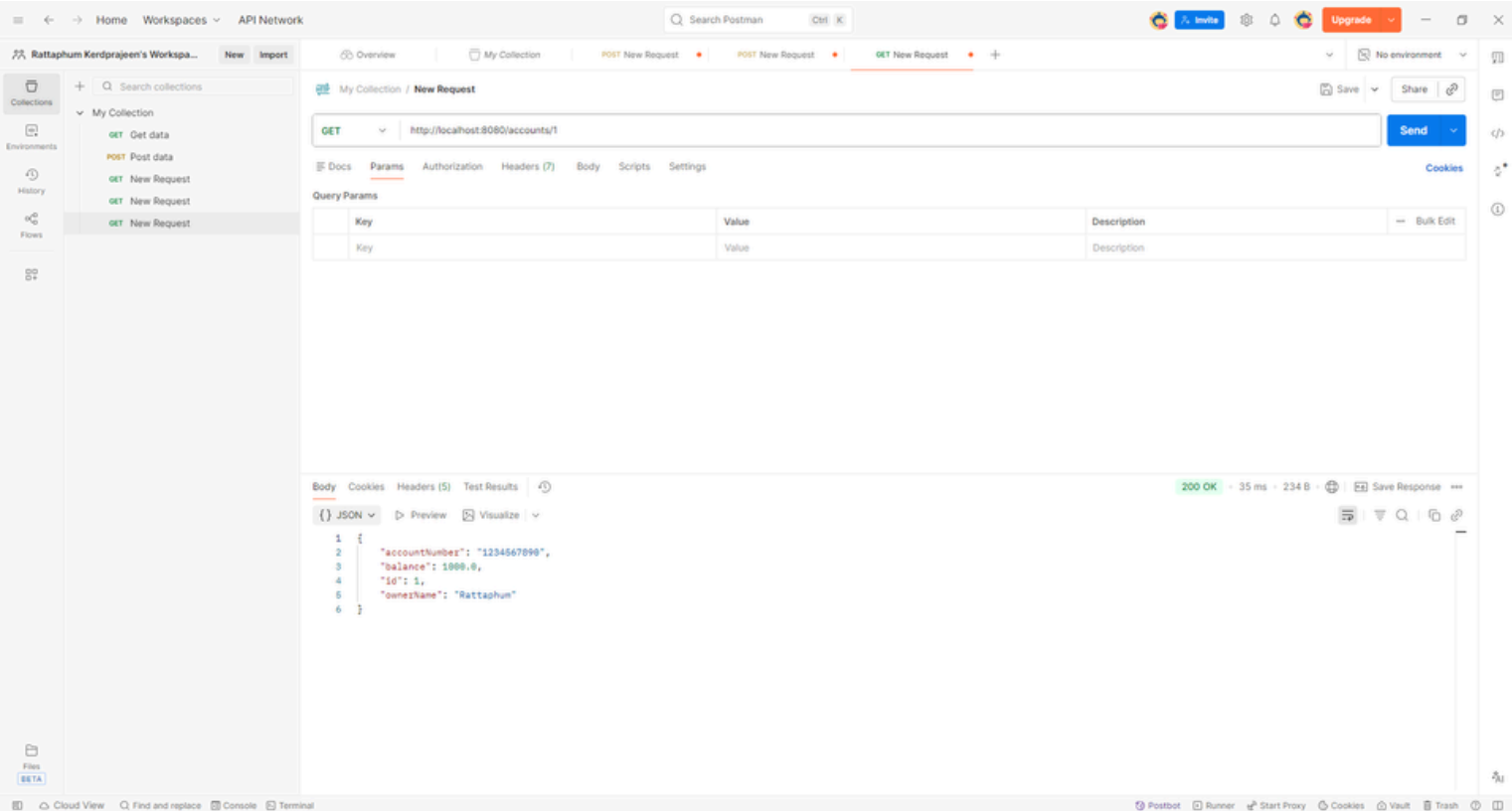




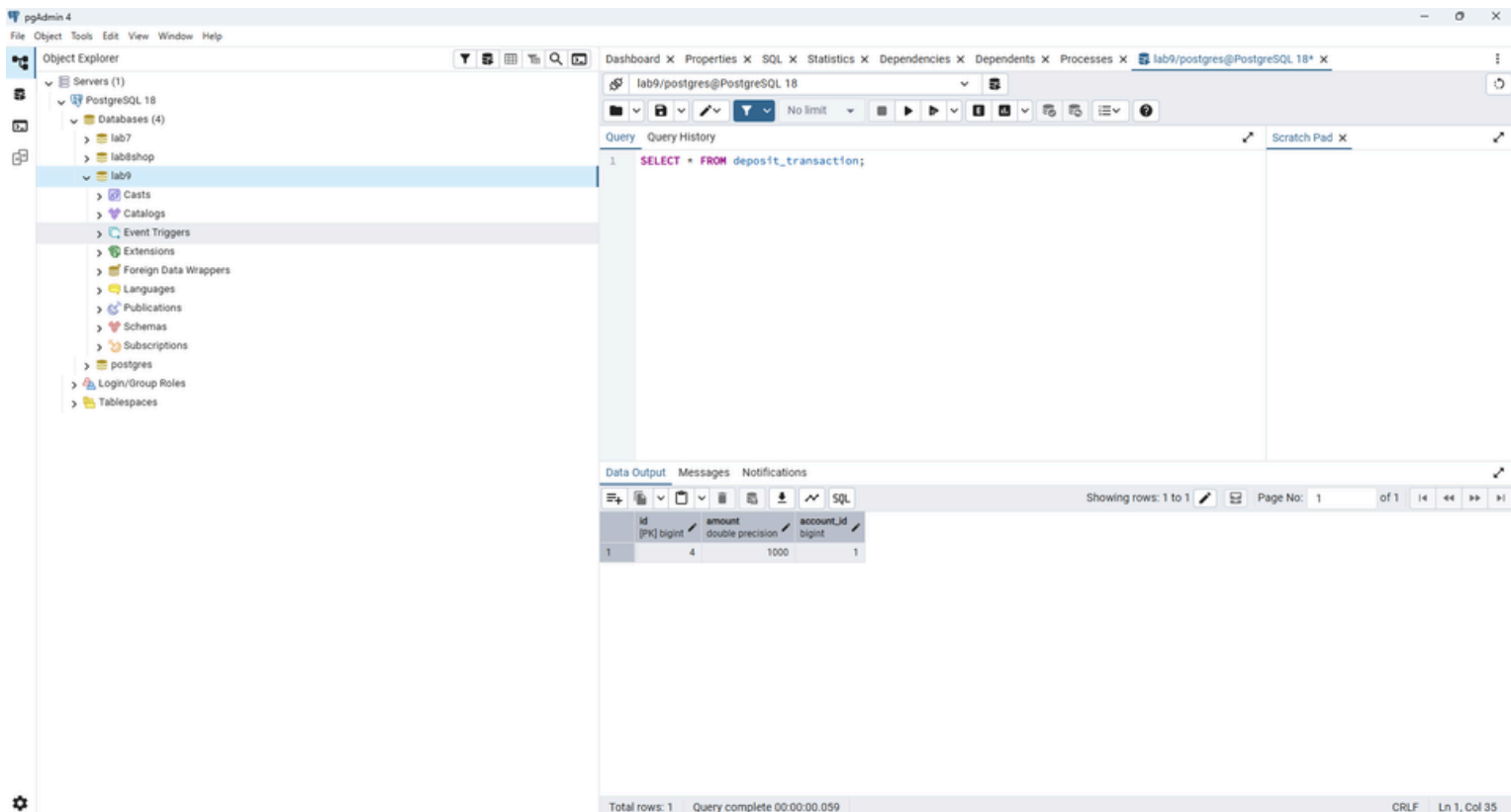
จากภาพเป็นการทดสอบการสร้างบัญชีธนาคารผ่าน Postman โดยส่ง Request แบบ POST ไปยัง <http://localhost:8080/accounts> พร้อมข้อมูลเลขบัญชี ชื่อเจ้าของบัญชี และยอดเงินเริ่มต้น 0 บาท ระบบสามารถสร้าง Account ได้สำเร็จและส่งข้อมูลกลับมาเป็น Response พร้อมกับ id ของบัญชี ซึ่งแสดงว่าข้อมูลถูกบันทึกลงในฐานข้อมูลเรียบร้อยแล้ว



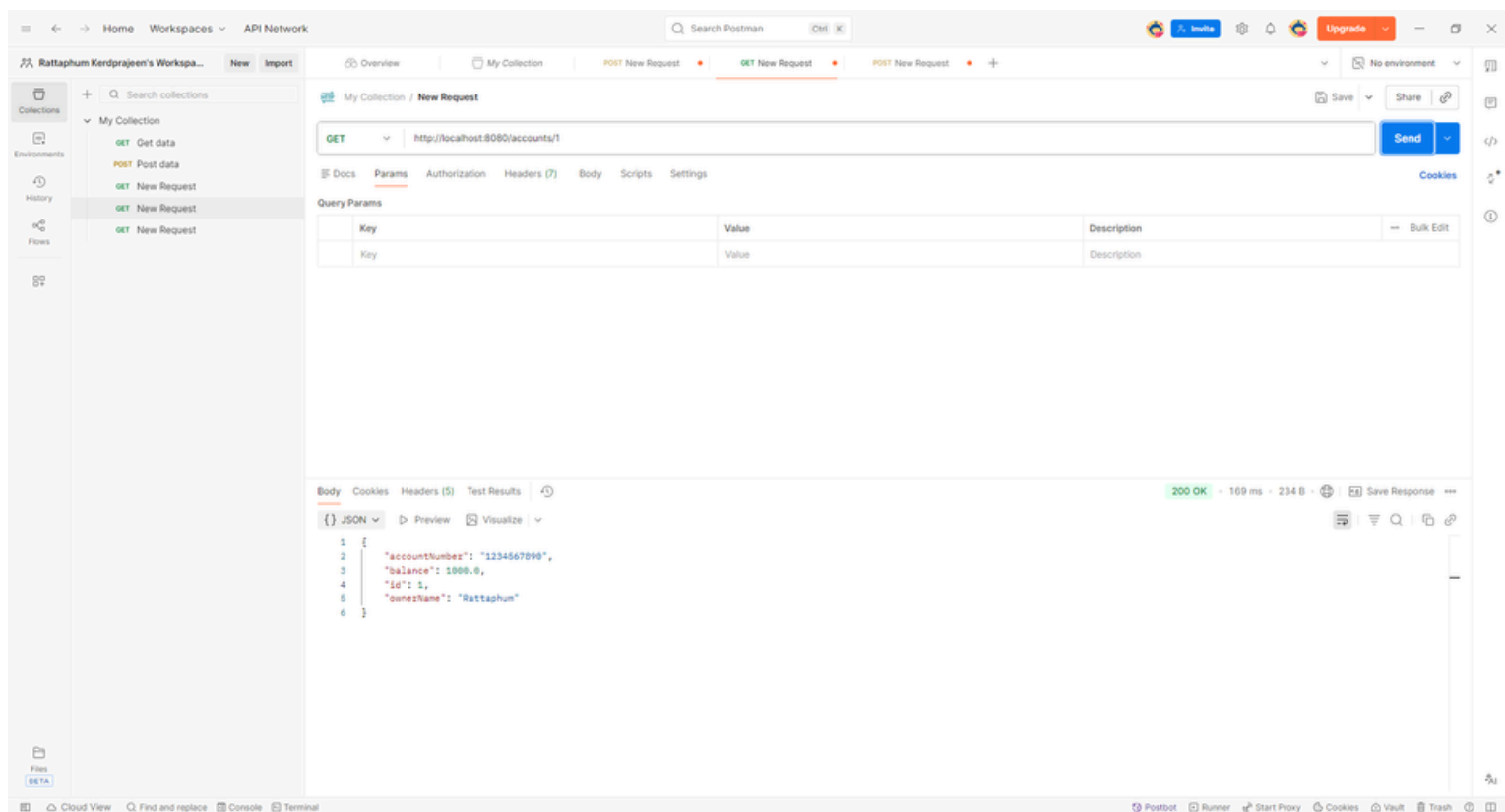
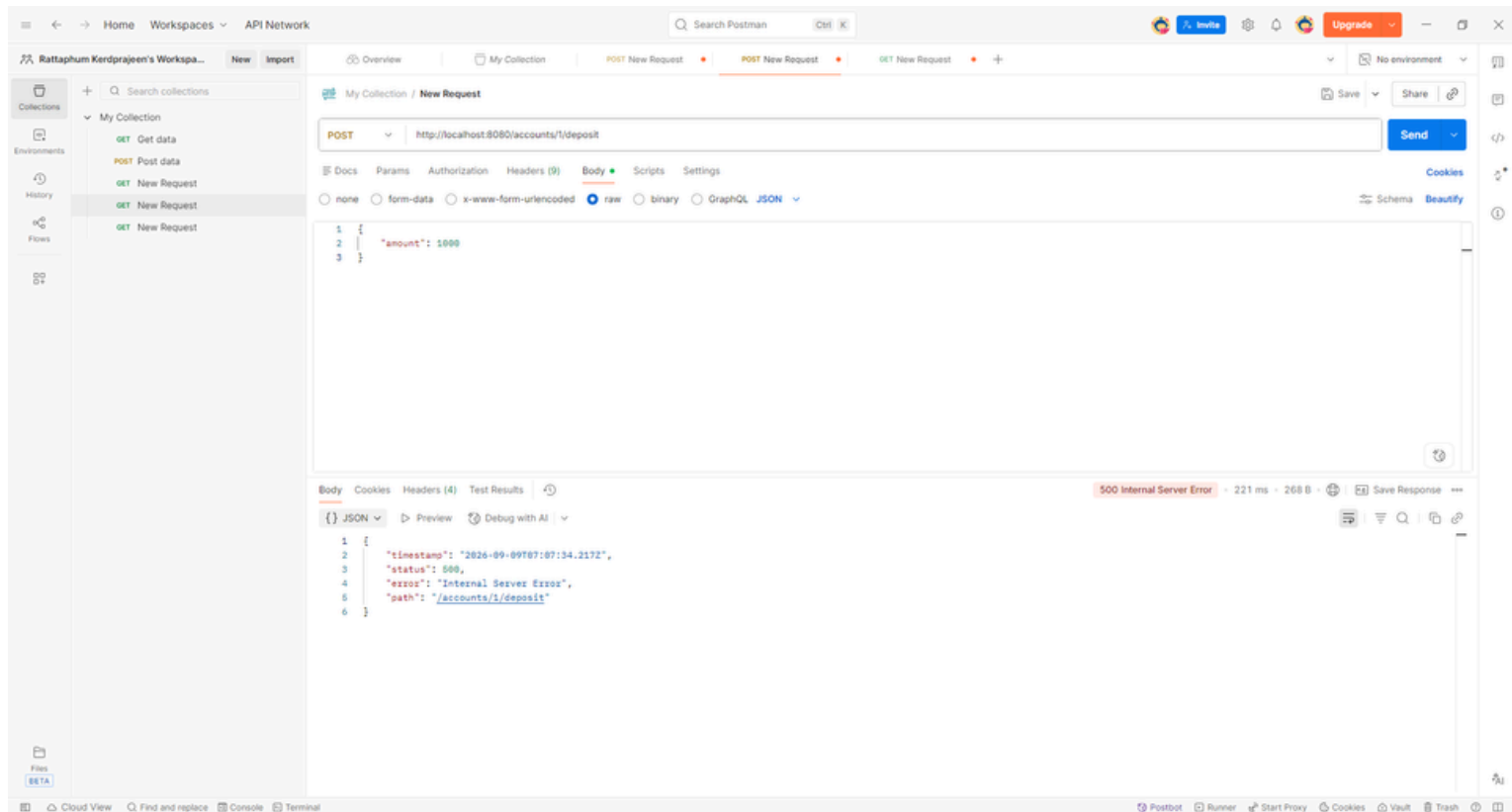
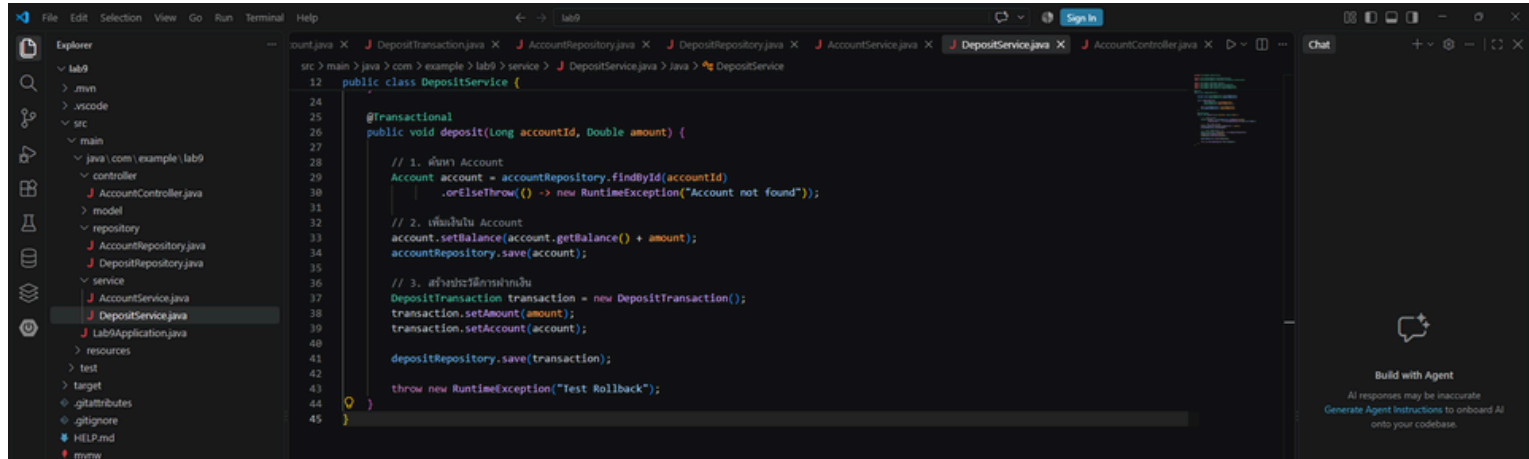
จากภาพเป็นการทดสอบการฝากเงินผ่าน Postman โดยส่ง Request แบบ POST ไปยัง <http://localhost:8080/accounts/{id}/deposit> และส่งจำนวนเงินที่ต้องการฝากในรูปแบบ JSON จำนวน 1,000 บาท ระบบตอบกลับด้วยข้อความ Deposit successful แสดงว่าการฝากเงินทำงานสำเร็จ โดยระบบจะเพิ่มยอดเงินใน Account และบันทึกประวัติการฝากเงินลงในฐานข้อมูลภายใน Transaction เดียวกัน

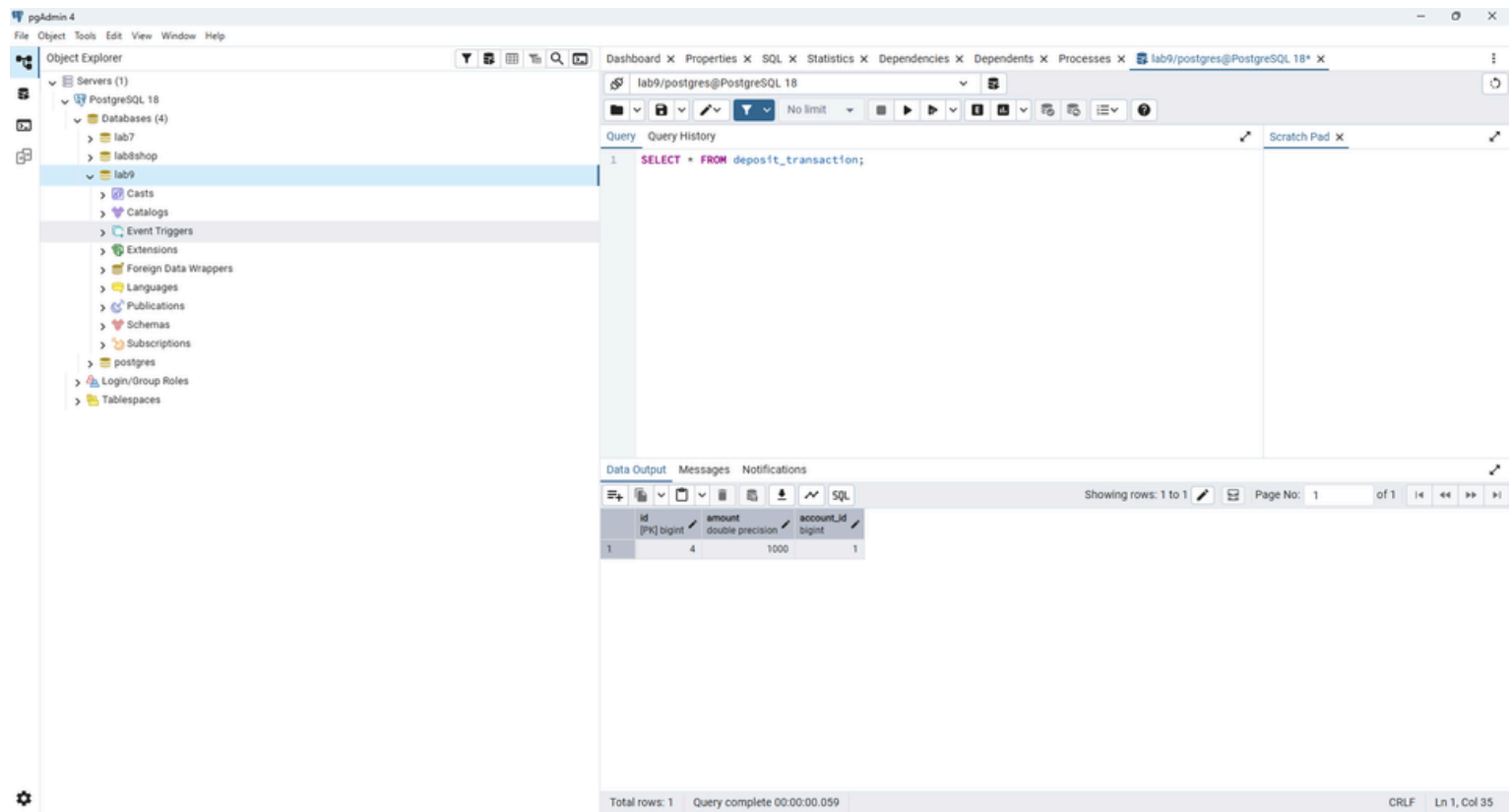


จากภาพเป็นการตรวจสอบข้อมูล Account หลังจากทำการฝากเงิน โดยใช้ Request แบบ GET ไปยัง `http://localhost:8080/accounts/{id}` ผลลัพธ์แสดงข้อมูลของบัญชีและพบว่ายอดเงิน balance เปลี่ยนจาก 0 บาท เป็น 1,000 บาท แสดงว่าระบบสามารถเพิ่มยอดเงินในบัญชีได้ถูกต้องหลังจากทำรายการฝากเงินสำเร็จ

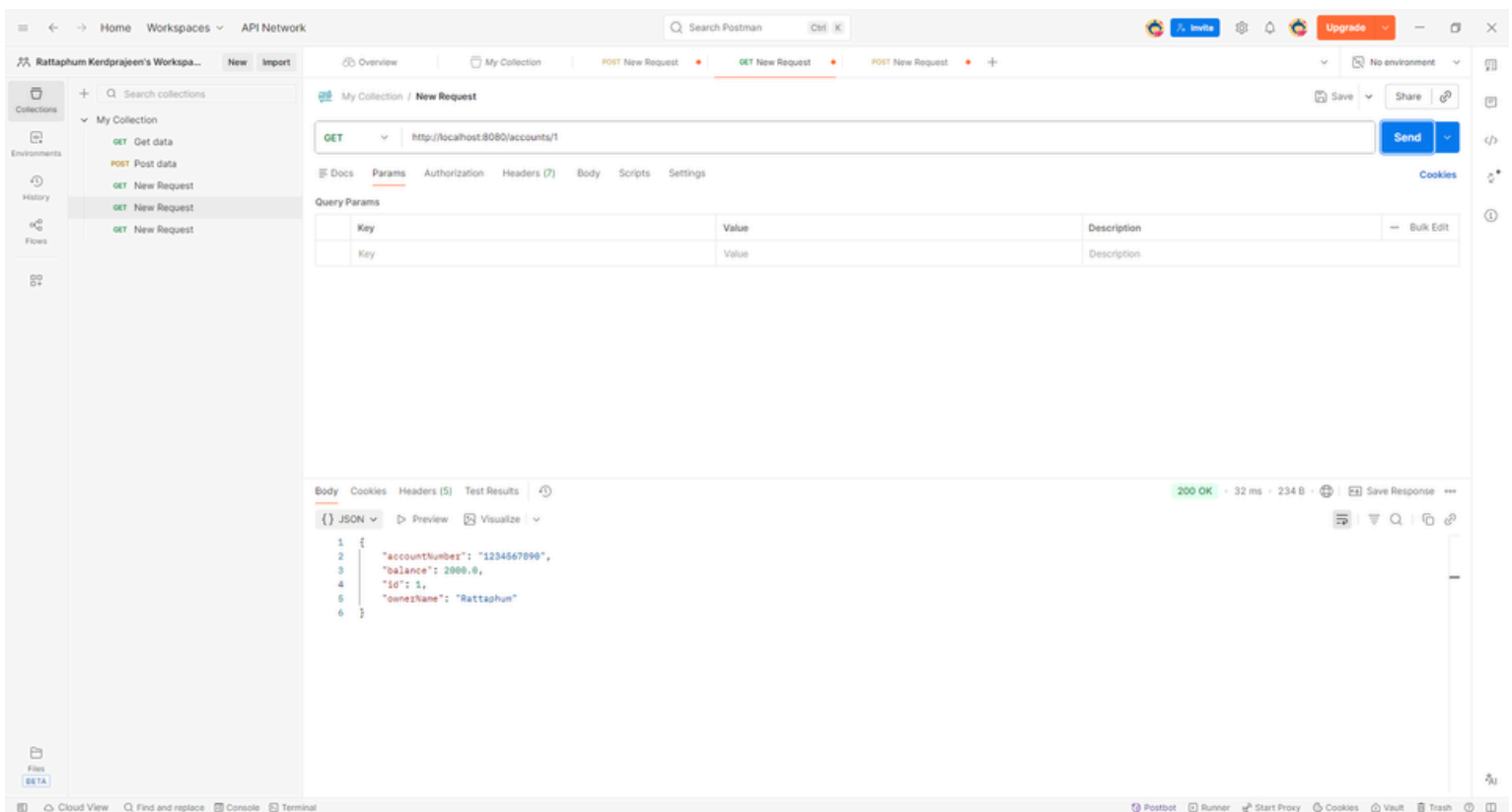
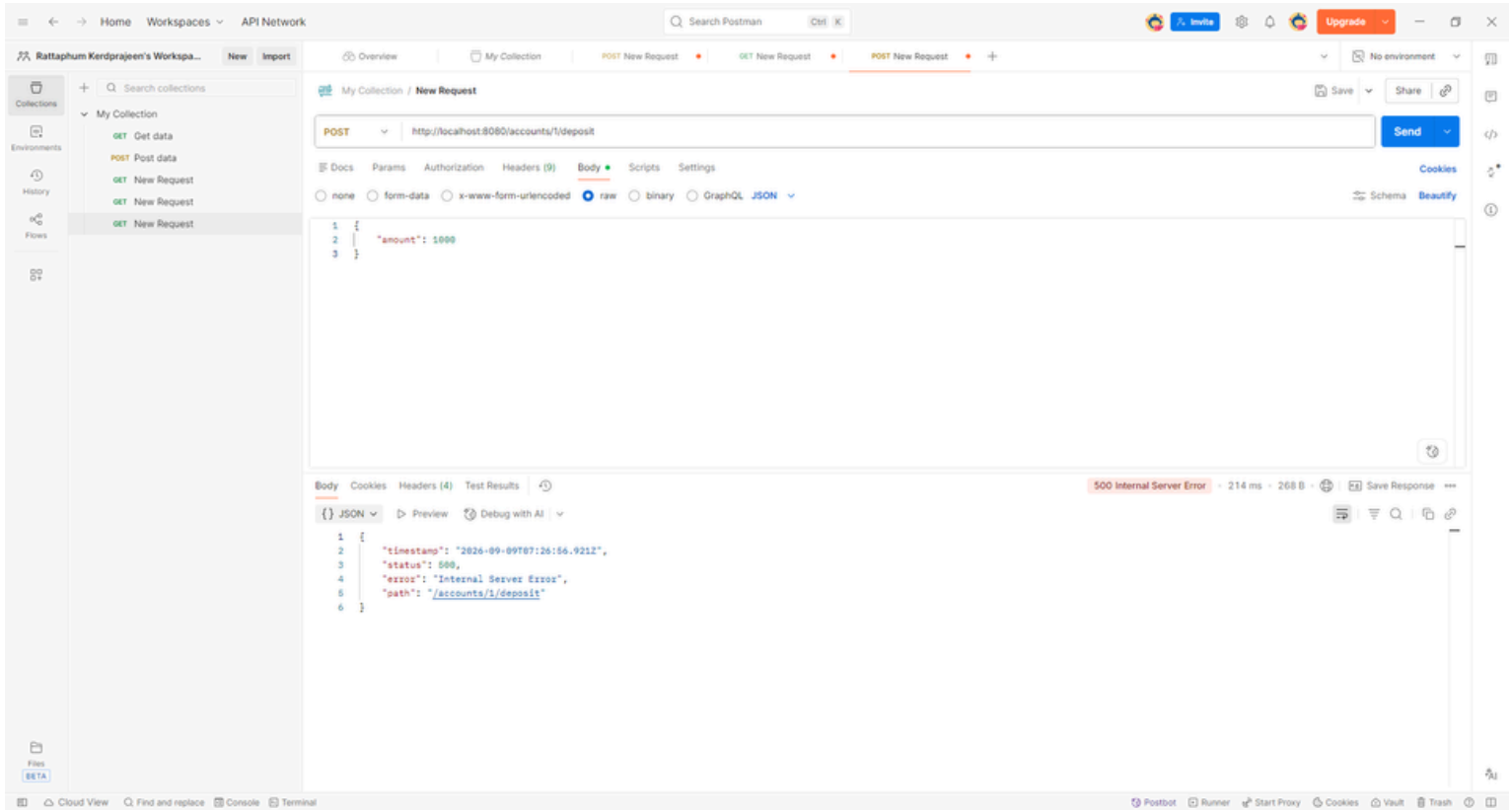
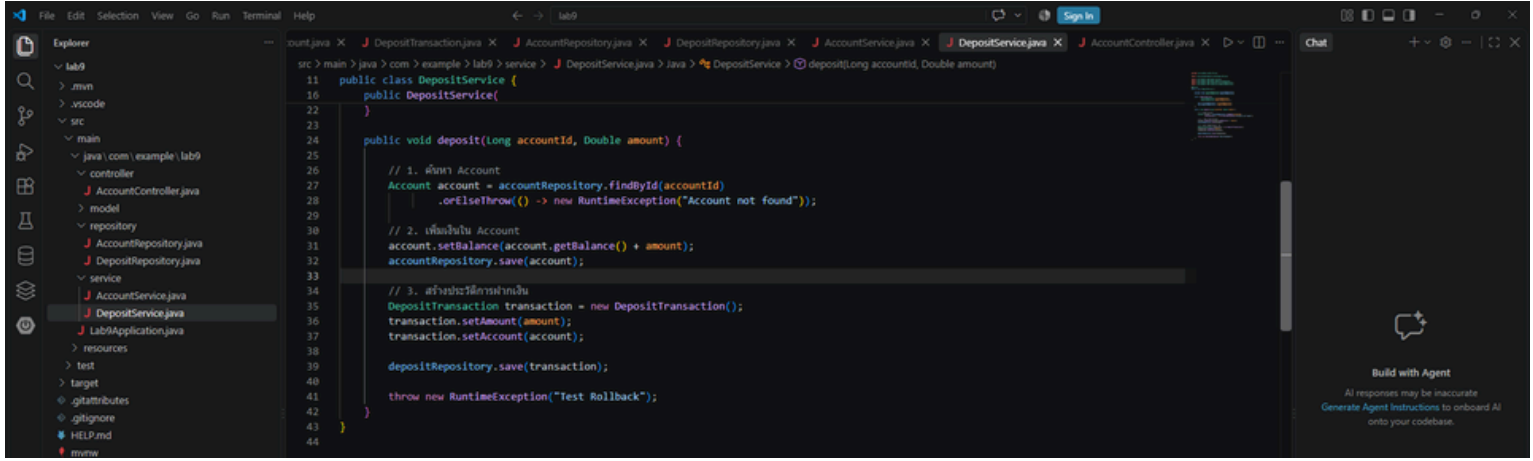


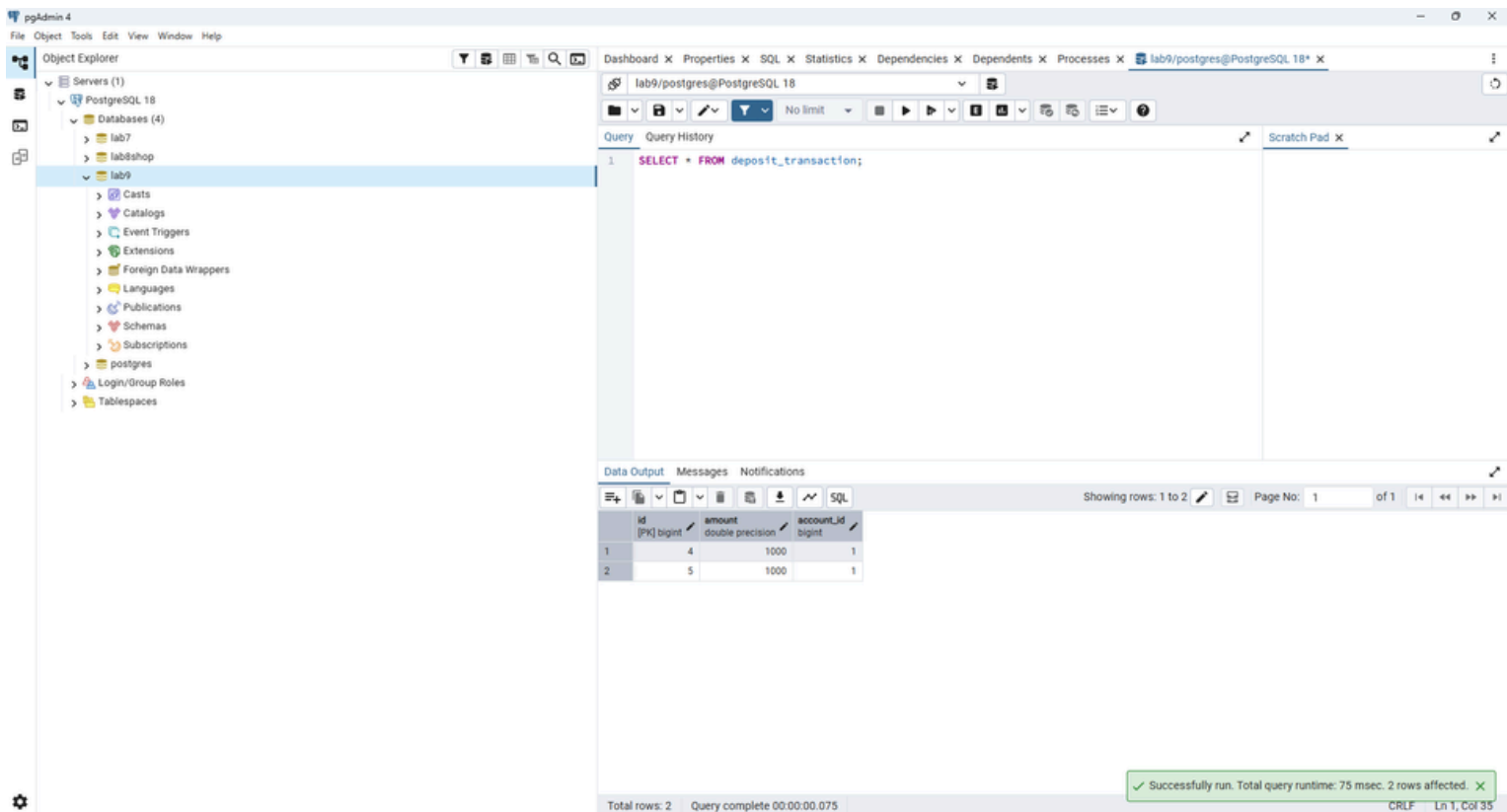
จากภาพเป็นการตรวจสอบตาราง deposit_transaction ใน PostgreSQL ผ่าน pgAdmin โดยใช้คำสั่ง `SELECT * FROM deposit_transaction;` ผลลัพธ์พบข้อมูลรายการฝากเงินจำนวน 1 รายการ โดยมีจำนวนเงิน 1,000 บาท และมี account_id ที่อ้างอิงไปยัง Account ที่ทำรายการ แสดงว่าระบบสามารถบันทึกประวัติการฝากเงินลงในฐานข้อมูลได้สำเร็จ



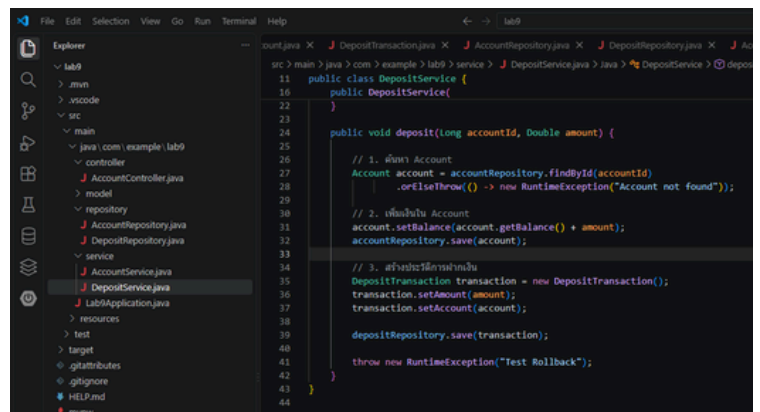
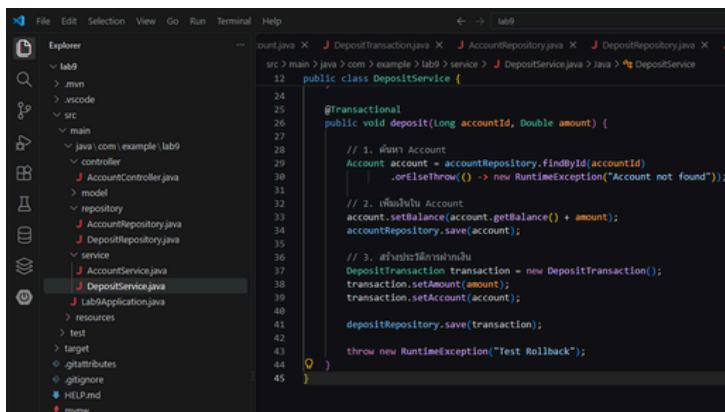


จากภาพเป็นการทดสอบ Transaction Rollback โดยได้เพิ่มคำสั่ง `throw new RuntimeException("Test Rollback");` หลังจากบันทึกข้อมูล Account และ DepositTransaction แล้ว เมื่อส่ง Request ฝากเงิน ระบบจึงเกิดข้อผิดพลาดและตอบกลับด้วย Status 500 Internal Server Error การทดลองนี้ใช้เพื่อจำลองสถานการณ์ที่เกิด Error หลังจากระบบดำเนินการบันทึกข้อมูลไปแล้ว





จากภาพเป็นการตรวจสอบผลหลังจากเกิด Error โดยตรวจสอบทั้งยอดเงินใน Account และข้อมูลในตาราง deposit_transaction พบว่ายอดเงินยังคงเป็น 1,000 บาท และจำนวนรายการฝากเงินยังเท่าเดิม ไม่มีรายการใหม่เพิ่มขึ้น แสดงว่า @Transactional สามารถทำงานแบบ ROLLBACK ได้สำเร็จ โดยยกเลิกการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นภายใน Transaction เมื่อเกิด Error



จากการทดลองพบว่าเมื่อมี @Transactional และเกิด Error ระบบจะทำการ ROLLBACK ทำให้ข้อมูลที่เปลี่ยนแปลงภายใน Transaction ถูกย้อนกลับ และข้อมูลในฐานข้อมูลยังคงเหมือนเดิม ในทางกลับกัน เมื่อเอา @Transactional ออกจากเมธอด deposit() แล้วเกิด Error ขึ้น ข้อมูลที่ถูกบันทึกไปก่อนเกิด Error ยังคงอยู่ในฐานข้อมูล เนื่องจากไม่มี Transaction ที่ควบคุมการทำงานทั้งหมดให้เป็นงานก้อนเดียวกัน ดังนั้นการใช้ @Transactional จึงช่วยให้การฝากเงินซึ่งประกอบด้วยหลายขั้นตอนสามารถสำเร็จพร้อมกัน หรือถูกยกเลิกพร้อมกันเมื่อเกิดข้อผิดพลาด

1.@Transactional มีหน้าที่อะไร?

- @Transactional มีหน้าที่กำหนดให้คำสั่งต่าง ๆ ภายในเมธอดทำงานอยู่ภายใต้ Transaction เดียวกัน โดยถ้าการทำงานทั้งหมดสำเร็จ ระบบจะทำ COMMIT เพื่อบันทึกข้อมูลลงฐานข้อมูล แต่ถ้าเกิด Error ระบบจะทำ ROLLBACK เพื่อยกเลิกการเปลี่ยนแปลงทั้งหมดภายใน Transaction นั้น

2.เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

- การฝากเงินควรใช้ Transaction เพราะการฝากเงินหนึ่งครั้งมีการเปลี่ยนแปลงข้อมูลมากกว่าหนึ่งอย่าง ได้แก่ การเพิ่มยอดเงินใน Account และการบันทึกประวัติการฝากเงิน ซึ่งทั้งสองขั้นตอนควรสำเร็จพร้อมกัน หากขั้นตอนใดเกิดข้อผิดพลาด ระบบควรยกเลิกการเปลี่ยนแปลงทั้งหมด เพื่อป้องกันไม่ให้ข้อมูลในฐานข้อมูลไม่สอดคล้องกัน

3.COMMIT และ ROLLBACK ต่างกันอย่างไร?

- COMMIT คือการยืนยันการเปลี่ยนแปลงข้อมูลทั้งหมดภายใน Transaction และบันทึกข้อมูลลงฐานข้อมูลอย่างถาวร เมื่อการทำงานสำเร็จ
- ROLLBACK คือการยกเลิกการเปลี่ยนแปลงข้อมูลทั้งหมดภายใน Transaction และย้อนกลับไปยังสถานะก่อนเริ่ม Transaction เมื่อเกิดข้อผิดพลาด

4.Account และ DepositTransaction มีความสัมพันธ์แบบใด?

- Account และ DepositTransaction มีความสัมพันธ์แบบ One-to-Many โดย Account หนึ่งบัญชีสามารถมี DepositTransaction ได้หลายรายการ ส่วน DepositTransaction แต่ละรายการจะอ้างอิงไปยัง Account หนึ่งบัญชีผ่าน Foreign Key ที่ชื่อ account id
- ในฝั่ง Entity ของ DepositTransaction จึงกำหนดความสัมพันธ์ด้วย @ManyToOne

5.เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

- เมื่อเกิด Error ภายใน Transaction ระบบจะทำ ROLLBACK ทำให้การเปลี่ยนแปลงข้อมูลทั้งหมดที่เกิดขึ้นภายใน Transaction ถูกยกเลิก แม้ว่าก่อนเกิด Error จะมีการเรียก save() เพื่อบันทึก Account และ DepositTransaction ไปแล้วก็ตาม ดังนั้นข้อมูลใน Database จะกลับไปอยู่ในสถานะก่อนเริ่ม Transaction

6.เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

- เมื่อมี @Transactional หากเกิด Error ระบบจะทำ ROLLBACK และยกเลิกการเปลี่ยนแปลงทั้งหมดที่อยู่ภายใน Transaction
- แต่เมื่อลบ @Transactional ออก การบันทึกข้อมูลแต่ละคำสั่งจะไม่ได้ถูกควบคุมให้เป็น Transaction เดียวกัน เมื่อเกิด Error หลังจากมีการบันทึกข้อมูลไปแล้ว ข้อมูลที่บันทึกสำเร็จก่อนเกิด Error จะยังคงอยู่ในฐานข้อมูล ทำให้ข้อมูลอาจไม่สอดคล้องกัน เช่น ยอดเงินใน Account ถูกเพิ่มขึ้นแล้ว แต่การทำงานส่วนถัดไปเกิดข้อผิดพลาด